CUESTIONARIO 02

Control de flujo

Pregunta 1

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

¿Cuál de las siguientes afirmaciones sobre el ciclo for mejorado introducido en Java 5 es verdadera?

Seleccione una:

a. El ciclo for mejorado solo se puede utilizar con matrices.

b. El ciclo for mejorado se puede utilizar con cualquier tipo de colección y matriz. ✓

c. El ciclo for mejorado permite la modificación de los elementos de la colección durante la iteración.

d. El ciclo for mejorado no requiere que la colección implemente la interfaz Iterable.

Justificación: El for-each funciona con arreglos y con cualquier objeto que implemente Iterable (todas las Collection). La d es falsa precisamente porque exige Iterable. La c también lo es: agregar o quitar elementos durante la iteración lanza ConcurrentModificationException.

Fuente: Java.pdf #6 · también aparece en: Simulación APX V1 #33

La respuesta correcta es: El ciclo for mejorado se puede utilizar con cualquier tipo de colección y matriz.

Mi justificación
Los ciclos for each permite utilizarse en cualquier colección y matriz (array). Es teoría básica.

Pregunta 2

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

What is the result value of w?

```
public class Main {
    public static void main(String[] args) {
        int a = 10;
        int b = 37;
        int z = 0;
        int w = 0;

        if (a == b) {
            z = 3;
        } else if (a > b) {
            z = 6;
        }

        w = 10 * z;

        System.out.println("El valor de w es: " + w);
    }
}
```

Seleccione una:

- a. 30
- b. 60
- c. 0 ✓

Mi justificación
Ninguna de la dos condiciones del if ni else if es verdadera así que el valor de z sigue siendo 0, y w = 10 * 0 es igual a 0.

Pregunta 3

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

Que muestra en pantalla el siguiente código?

```
public class LoopQuestion {
    public static void main(String[] args) {
        int count = 0;
        for (int i = 0; i < 10; i++) {
            if (i % 2 == 0) {
                count++;
            }
        }
        System.out.println("Count: " + count);
    }
}
```

Seleccione una:

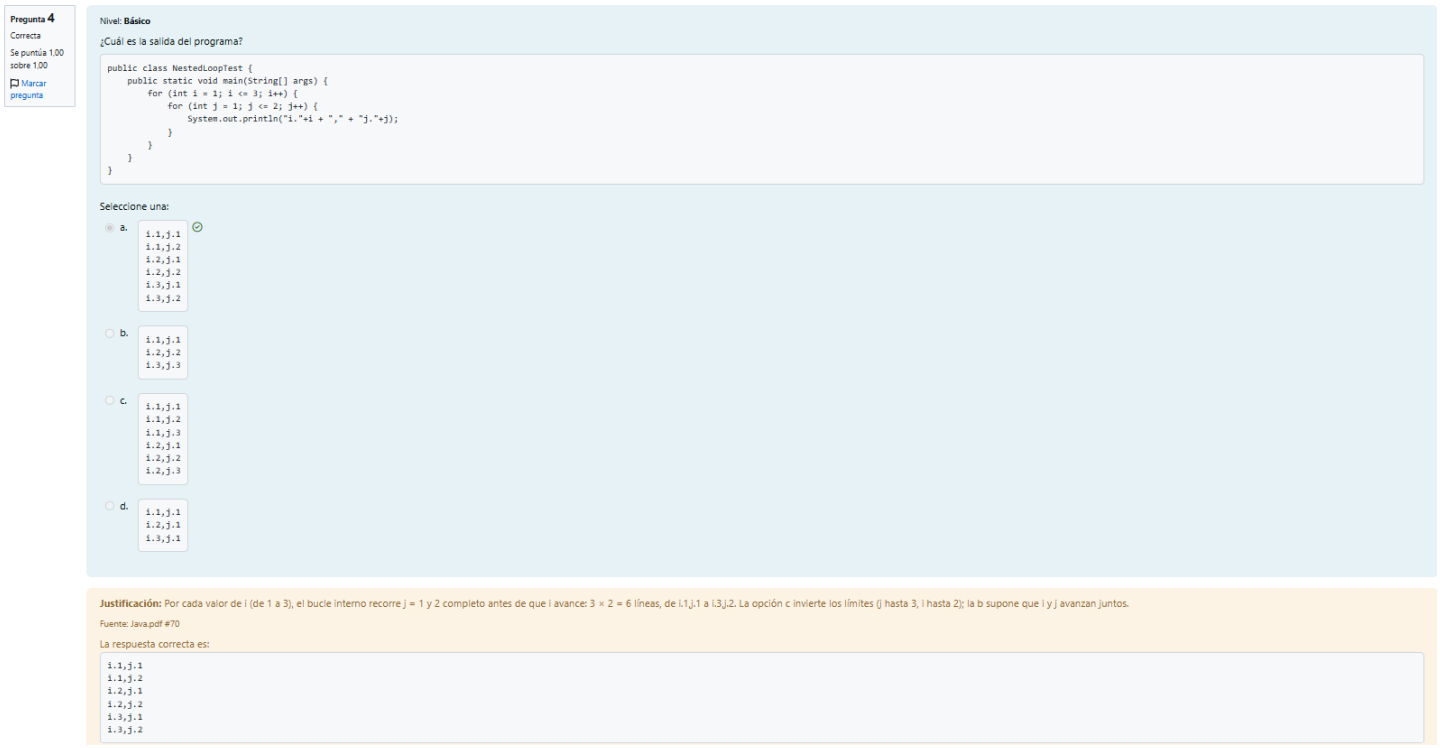
- a. Compilation fails.
- b. Count: 5 ✓
- c. Count: 4
- d. Count: 10
- e. Count: 6

Justificación: Los pares entre 0 y 9 son 0, 2, 4, 6 y 8: cinco en total. El distractor 4 olvida que el 0 también es par.

Fuente: Simulación APX V4 #15 · también aparece en: Simulación APX V1 #28, Java.pdf #40

La respuesta correcta es: Count: 5

Mi justificación
El ciclo solo ejecutará count++ con los números pares, y llega hasta i = 9 porque el segundo parámetro es i < 10, no i < 9, entonces se ejecutará cuando i valga 0, 2, 4, 6 y 8. Por tanto count = 5.



Mi justificación
El ciclo de i <= 3 se ejecutará con i = 1, i = 2, i = 3. El ciclo de j <= 2 se ejecutará con j = 1, j = 2. Haciendo el diagrama de árbol, se obtiene la opción a)

Pregunta 5

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

¿Cuántas veces se ejecutará la instrucción println más interna en el siguiente código?

```
public class NestedForLoopTest {
    public static void main(String[] args) {
        for (int i = 0; i < 4; i++) {
            for (int j = 0; j < 3; j++) {
                for (int k = 0; k < 2; k++) {
                    System.out.println("Hello");
                }
            }
        }
    }
}
```

Seleccione una:

- a. 6
- b. 12
- c. 24 ✓
- d. 36

Justificación: Los bucles anidados multiplican sus iteraciones: 4 (i) × 3 (j) × 2 (k) = 24 ejecuciones del println. El distractor 12 omite uno de los factores.

Fuente: Cuestionario APX V3 #27 · también aparece en: Java.pdf #135

La respuesta correcta es: 24

Mi justificación
El ciclo de i va de 0 a 3, ósea 4 vueltas El ciclo de j va de 0 a 2, ósea 3 vueltas El ciclo de k va de 0 a 1, ósea 2 vueltas 4 * 3 * 2 = 24 veces se imprime “Hello”

Pregunta 6

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

What is the result?

```
public class DoWhile {
    public static void main(String[] args) {
        int ii = 2;
        do {
            System.out.println(ii);
        } while (ii > 0);
    }
}
```

Seleccione una:

a. 1 0

b. 2 1 2

c. Compilation fails

d. An infinite loop

Justificación: ii nunca se modifica dentro del do-while, así que la condición ii > 0 siempre es verdadera y el programa imprime 2 indefinidamente. El código compila sin problemas, por eso 'Compilation fails' es incorrecto.

Fuente: Java E #9

La respuesta correcta es: An infinite loop

Mi justificación
Ya que no hay nada que afecte el valor de ii dentro del do, su valor permanecerá igual cada vez que regrese al while así que siempre evaluará 2 > 0 así que crearía un ciclo infinito.

Pregunta 7

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

What is the result?

```
public class DoWhile {
    public static void main(String[] args){
        int ii = 2;
        do{
            System.out.println(ii);
        }while(--ii);
    }
}
```

Seleccione una:

a. 1 0

b. Compilation fails. ✓

c. 2 1 2

d. null

e. An infinite loop

Mi justificación
Lo que está dentro de while debe ser una comparación, --ii es una operación aritmética así que la compilación falla por eso.

Pregunta 8

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

What value should replace kk in line x to cause jj=5 to be output?

```
public class MyFive {
    public static void main(String[] args) {
        short ii;
        short jj = 0;
        for (ii = kk; ii > 6; ii -= 1) { // line x
            jj++;
        }
        System.out.println("jj=" + jj);
    }
}
```

Seleccione una:

- a. -1
- b. 1
- c. 5
- d. 8
- e. 11 ✓

Justificación: Con kk = 11 el bucle corre para ii = 11, 10, 9, 8 y 7 (5 iteraciones) y se detiene en 6, así que imprime jj=5. Con 8 solo hay 2 iteraciones, y con 5, 1 o -1 ninguna (ii > 6 es falso desde el principio). Compila con short porque un literal int constante y representable se puede asignar a short, y 'ii -= 1' incluye un cast implícito.

Nota sobre el material original: En el formulario original el código ya declaraba short kk = 11, lo que regalaba la respuesta y no casaba con el enunciado (que habla de reemplazar kk); se restauró la forma de la pregunta OCA original, con kk como hueco en la línea x.

Fuente: Java E #10

La respuesta correcta es: 11

Mi justificación
Si ii = 11, entonces el ciclo se ejecutará cuando ii valga de 11 a 7, lo cual son 5 veces, eso hará que jj++ se ejecute 5 veces y obtengamos jj = 5.

Pregunta 9

Incorrecta

Se puntúa 0,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

¿Qué imprime este bucle?

```
public static void main(String[] args) {
    int num = 0;
    int arr[] = {1,2,3,4,5};
    for(int i = 0; i < arr.length; i++) {
        if(i % 2 == 0) {
            num += arr[i];
        } else {
            num -= arr[i];
        }
    }
    System.out.println(num);
}
```

Seleccione una:

- a. 0
- b. 3
- c. 5
- d. -3 ⊗

Justificación: En los índices pares se suma y en los impares se resta: 1 - 2 + 3 - 4 + 5 = 3. El distractor 5 resulta de pensar solo en el último elemento, y -3 de invertir los signos (la condición usa el índice i, no el valor).

Fuente: Quiz APX #4

La respuesta correcta es: 3

Mi justificación
El código inicia con num = 0 y sumará los pares mientras que restará los impares 0 – 1 + 2 – 3 + 4 – 5 = -3 (Era al revés)

Pregunta 10

Correcta

Se puntúa 1.00 sobre 1.00

Marcar pregunta

Nivel: **Intermedio**

What is the result if the integer value is 33?

```
public static void main(String[] args) {
    int value = 33;

    if (value >= 0) {
        if (value != 0) {
            System.out.println("the ");
        } else {
            System.out.println("quick ");
        }

        if (value < 10) {
            System.out.println("brown ");
        }

        if (value > 30) {
            System.out.println("fox");
        } else if (value < 50) {
            System.out.println("jumps ");
        } else if (value < 10) {
            System.out.println("over ");
        } else {
            System.out.println("the ");
        }

        if (value > 10) {
            System.out.println("lazy ");
        } else {
            System.out.println("dog ");
        }

        System.out.println("...");
    }
}
```

Seleccione una:

a. The fox lazy... ✓

b. The fox jump lazy...

c. Quick fox over lazy...

d. Quick fox the...

Justificación: 33 != 0 imprime 'the', y 33 < 10 es falso. En la cadena if/else-if, 33 > 30 es verdadero, así que se imprime 'fox' sin evaluar las demás ramas. 33 > 10 imprime 'lazy' y al final se imprime '...'. La opción b es el distractor de quien evalúa también 'value < 50' sin notar que un else-if se salta cuando una rama anterior ya se cumplió.

Fuente: Java E #16 · también aparece en: Java Sesiones #28

La respuesta correcta es: The fox lazy...

Mi justificación
Como value = 33, el primer if es true porque 33>=0 e imprime the. El segundo if es true igual porque 33>30 e imprime fox. El segundo if es true igual porque 33>10 e imprime lazy. Al final quedamos con the fox lazy.

Pregunta 11

Incorrecta

Se puntúa 0,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

What is the result?

```
public static void main(String[] args) {
    String color = "Red";
    switch (color) {
    case "Red":
        System.out.println("Found Red");
    case "Blue":
        System.out.println("Found Blue");
    case "White":
        System.out.println("Found White");
        break;
    default:
        System.out.println("Found Default");
    }
}
```

Seleccione una:

- a. Found Red.
- b. Found Red Found Blue.
- c. Found Red Found Blue Found White.
- d. Found Red Found Blue Found White Found Default. ⊗

Justificación: "Red" coincide con el primer case y no hay break, así que la ejecución cae (fall-through) por "Blue" y "White" hasta el break. Imprime Found Red, Found Blue y Found White. El distractor d ignora el break que hay antes de default.

Fuente: Java Sesiones #13

La respuesta correcta es: Found Red Found Blue Found White.

Mi justificación
El case “Red” se ejecuta e imprime “Found Red”, como no tiene break ninguno otro, se ejecutan todos. Quedando Found Red Found Blue Found White Found Default. (No vi el break en White)

Pregunta 12

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

What is the result?

```
enum Days {
    MONDAY, TUESDAY, WEDNESDAY, THURSDAY
}
public class Test {
    public static void main(String[] args) {
        int y = 1;
        Days d = Days.TUESDAY;
        switch (d) {
            case MONDAY: y++;
            case TUESDAY: y += 10;
            case WEDNESDAY: y += 100;
            case THURSDAY:  y += 1000;
        }
        System.out.println(y);
    }
}
```

Seleccione una:

- a. 11
- b. 111
- c. Compilation fails.
- d. An exception is thrown at runtime.
- e. 1111 ✓

Justificación: Sin break, desde case TUESDAY se produce fall-through por todos los case siguientes: 1 + 10 + 100 + 1000 = 1111. La opción a (11) supone que el switch se detiene tras el case que coincide. En un switch sobre enum, los case usan el nombre de la constante sin calificar, así que compila.

Fuente: Java.pdf #112 · también aparece en: Simulación APX V4 #39

La respuesta correcta es: 1111

Mi justificación
El switch cae en el caso de TUESDAY así que se ejecuta y += 10, y como no tiene break, ejecuta todos los demás posteriores, entonces 1 + 10 + 100 + 1000 = 1111.

Pregunta 13

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

Que se imprime en pantalla al ejecutar el siguiente código?

```
public class Test {
    public static void main(String... arguments) {
        final char a = 'A', d = 'D';
        char grade = 'B';
        switch (grade) {
            case a:
            case 'B':
                System.out.print("B");
            case 'C':
                System.out.print("C");
            case d:
            case 'F':
                System.out.print("F");
        }
    }
}
```

Seleccione una:

- a. A
- b. BF
- c. Compilation fails
- d. C
- e. ACF
- f. BCF ✓
- g. B

Justificación: a y d son variables locales final inicializadas con literales, es decir, constantes de compilación, y por eso valen como etiquetas case. grade = 'B' entra en case 'B' y, al no haber break, cae a los case siguientes: imprime B, C y F. El distractor "Compilation fails" supone que case a no es constante.

Fuente: Simulación APX V4 #4 · también aparece en: Java.pdf #14

La respuesta correcta es: BCF

Mi justificación
El switch cae en case 'B' así que imprime "B", y como no hay breaks después, se ejecutan los siguientes casos, los cuales sí compilan bien ya que la variable d es final. Entonces nos quedamos con BCF.

Pregunta 14

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

What is the result?

```
public class Test {
    public static void main(String... arguments) {
        final char a = 'A', d = 'D';
        char grade = 'B';
        switch (grade) {
            case a:
            case 'B':
                System.out.print("B");
            case 'C':
                System.out.print("C");
                break;
            case d:
            case 'F':
                System.out.print("F");
        }
    }
}
```

Seleccione una:

- a. B
- b. Compilation fails
- c. C
- d. A
- e. AC
- f. BC ✓

Justificación: a y d son variables locales final inicializadas con literales, es decir, constantes de compilación, así que valen como etiquetas case. grade = 'B' entra en case 'B', imprime B y, por fall-through, imprime C hasta llegar al break. El distractor «Compilation fails» supone que no se pueden usar variables en un case.

Fuente: Java.pdf #139 · también aparece en: Cuestionario APX V3 #30

La respuesta correcta es: BC

Mi justificación
El switch cae en case 'B' así que imprime "B", como no tiene break también se imprime el siguiente caso "C", y como este sí tiene break, se termina hasta ahí el switch. Nos quedamos con BC.

Pregunta 15

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

Que muestra en pantalla el siguiente código?

```
public class Test {
    public static void main(String[] args) {
        int x = 0;
        while (x++ < 5) {
            // Empty loop body
        }
        String message = x > 5 ? "greater than" : "false";
        System.out.println(message + " - " + x);
    }
}
```

Seleccione una:

a. false than - 6
b. greater than - 6 ✓
c. false than - 5
d. Compilation fails.
e. greater than - 5

Justificación: El post-incremento compara y después incrementa: la última comparación es 5 < 5 (false) y aun así x pasa a 6. Como 6 > 5, message = "greater than". El distractor "- 5" olvida el incremento de la comparación que termina el bucle.

Fuente: Simulación APX V4 #16 · también aparece en: Cuestionario APX V3 #11, Java.pdf #161, Java.pdf #41

La respuesta correcta es: greater than - 6

Mi justificación
Como x inicia con x = 0, x++ se ejecutará después de evaluar la condición así que llegará hasta 5 < 5 y justo después se convertirá en 6. Ahora, como message pide si x > 5 y x = 6 entonces es true y le asigna el primer valor. Entonces message es “greater than”. Qudamos con “greater than -6”

Pregunta 16

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

How many times is 2 printed?

```
public static void main(String[]args){
    String[] table = {"aa", "bb", "cc"};
    int ii = 0;
    for (String ss : table){
        while (ii<table.length){
            System.out.println(ii); ii++;
            break;
        }
    }
}
```

Seleccione una:

- a. Thrice
- b. It is not printed because compilation fails
- c. Twice
- d. Zero
- e. Once ✓

Justificación: En cada vuelta del for, el while imprime ii una sola vez y el break sale del while, no del for. Se imprimen 0, 1 y 2, así que el 2 aparece una vez. El distractor «Thrice» confunde las tres vueltas del for con las veces que se imprime el 2.

Fuente: Java D #5 · también aparece en: Java Sesiones #49

La respuesta correcta es: Once

Mi justificación
El for se ejecutará 3 veces porque table tiene 3 elementos. El while se ejecutará 2 veces, cuando ii vaya de 0 a 2. Así, se imprime el 2 cuando ii = 2 y se ejecute system.println(ii). Se imprime una sola vez

Pregunta 17

Incorrecta

Se puntúa 0,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

Cual es el resultado?

```
public class Test {
    static boolean b;
    public static void main(String[] args) {
        int x = 0;
        if (b) {
            x = 1;
        } else if (b = false) {
            x = 2;
        } else if (b) {
            x = 3;
        } else {
            x = 4;
        }
        System.out.println("x=" + x);
    }
}
```

Seleccione una:

- a. x = 3
- b. x = 0
- c. Compilation fails ⊗
- d. x = 4
- e. x = 1

Justificación: b es un campo static boolean, así que vale false por defecto. En else if (b = false) hay una ASIGNACIÓN, no una comparación: b queda en false y la condición vale false. El siguiente if (b) también es false, así que se ejecuta el else y x = 4. El distractor x = 2 lee = como ==, y «Compilation fails» no aplica porque la asignación devuelve un boolean.

Fuente: Simulación APX V4 #11 · también aparece en: Java.pdf #26

La respuesta correcta es: x = 4

Mi justificación
La compilación falla porque b = false no puede ser una condición de if en el caso de else if (b = false).

Pregunta 18

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

Which one is valid as a replacement for foo?

```
public static void main(String[] args) {
    Boolean b1 = true;
    Boolean b2 = false;
    int i = 0;
    while (foo) { }
}
```

Seleccione una:

a. b1.compareTo(b2)

b. i = 1

c. i == 2? -1:0

d. "foo".equals("bar") ✓

Justificación: La condición de un while debe ser boolean. "foo".equals("bar") llama a equals sobre un literal String y devuelve boolean (false), así que es el único reemplazo válido. compareTo devuelve int, «i = 1» es una asignación de tipo int (Java no convierte int a boolean) y el ternario i == 2 ? -1 : 0 también devuelve int.

Nota sobre el material original: En el formulario original la opción d aparecía como foo.equals("bar"), sin comillas en foo, y así ninguna opción compilaba; se restauraron las comillas como en la pregunta OCA original. La clave del examen (d) no cambia.

Fuente: Java Sesiones #46

La respuesta correcta es: "foo".equals("bar")

Mi justificación
La condición de while debe ser un true o false o una comparación que regrese esos valores, entonces. XXX La opción a) con compareTo regresa -1, 0 ó 1. La opción b) es un entero. La opción c) regresaría -1 ó 0. Ninguna es booleana VVV La opción d) si regresaría un booleano, en este caso false.

Pregunta 19

Incorrecta

Se puntúa 0,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

¿Que imprime en pantalla el código anterior?

```
public class Test {
    static int co = 0;
    public static void main(String[] args) {
        int x = 1;
        for (test('1'); test('2') && (x <= 2); test('3')) {
            x++;
            test('4');
        }
        System.out.println(" The co value = " + co);
    }
    static boolean test(char num) {
        System.out.print("" + num); co++; return true;
    }
}
```

Seleccione una:

- a. 12432432 The co value = 8
- b. 1234 The co value = 4
- c. 14231423 The co value = 8
- d. 12341234 The co value = 8
- e. 12342342 The co value = 8
- f. Compilation fails ⊗

Justificación: El for se ejecuta en este orden: inicialización (1), condición (2), cuerpo (4), actualización (3) y otra vez condición. Con x = 1 y x = 2 el cuerpo corre dos veces: 1 2 4 3 2 4 3. La tercera condición imprime 2 antes de que x <= 2 dé false, así que la salida es 12432432, con 8 llamadas a test. Los distractores ponen la actualización antes del cuerpo.

Fuente: Simulación APX V1 #22 · también aparece en: Simulación APX V4 #21, Java.pdf #57

La respuesta correcta es: 12432432 The co value = 8

Mi justificación
La compilación falla porque el segundo parámetro de for debe ser in intervalo numérico.

Pregunta 20

Incorrecta

Se puntúa 0.00 sobre 1.00

Marcar pregunta

Nivel: **Avanzado**

What values of x, y, z will produce the following result?

1 2 3 4
1 2 3 4
1 2 3 4
---
1 2 3 4

```
public static void main(String[] args){
    //insert code here
    int j = 0, k = 0;
    for (int i = 0; i<x; i++){
        do{
            k = 0;
            while (k<z){
                k++;
                System.out.print(k + " ");
            }
            System.out.println(" ");
            j++;
        }while (j<y);
        System.out.println("---");
    }
}
```

Seleccione una:

- a. int x = 4, y = 3, z = 2;
- b. int x = 2, y = 3, z = 3;
- c. int x = 3, y = 2, z = 3; ⊗
- d. int x = 2, y = 3, z = 4;
- e. int x = 4, y = 2, z = 3;

Justificación: z fija cuántos números hay por línea: «1 2 3 4» implica z = 4. En la primera vuelta del for, el do-while se repite hasta que j llega a y e imprime 3 líneas, así que y = 3. En la segunda vuelta j ya es ≥ y, pero el do-while se ejecuta una vez porque siempre corre al menos una. Hay dos «---», así que x = 2. El distractor a (4, 3, 2) invierte x y z.

Fuente: Java DE #22 · también aparece en: Java E #3, Java Sesiones #40

La respuesta correcta es: int x = 2, y = 3, z = 4;

Mi justificación
Como 1234 debe imprimirse 3 veces al inicio, la única opción que hace eso es si x = 3, así que la c) es la correcta.

RESULTADO = 16/20 = 8.0